

Object-Oriented System Analysis, Design, and Development

Phạm Thị Kim Ngôn

ngon.phamthikim@hoasen.edu.vn

Getting Started

Objectives

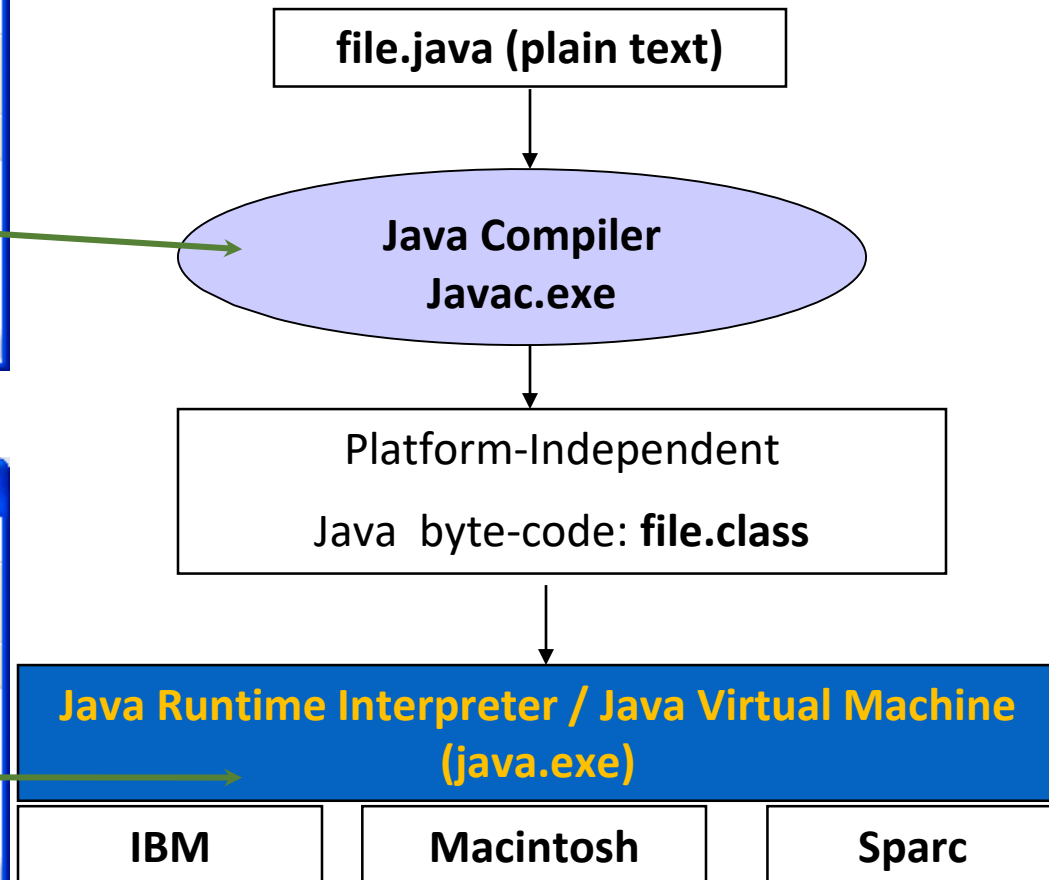
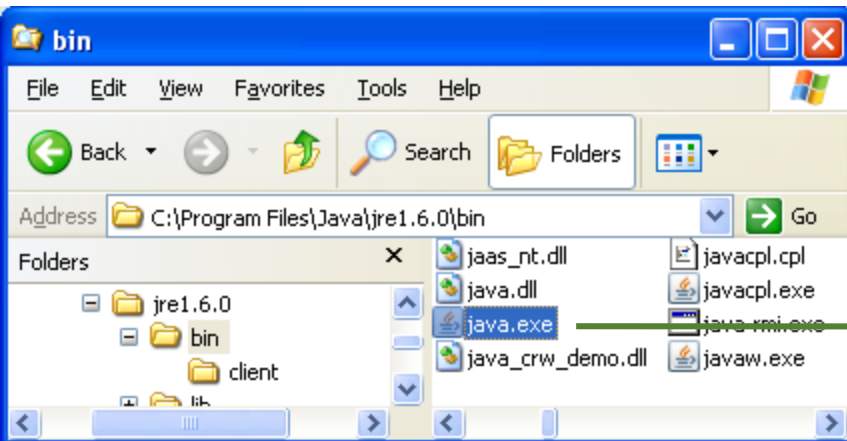
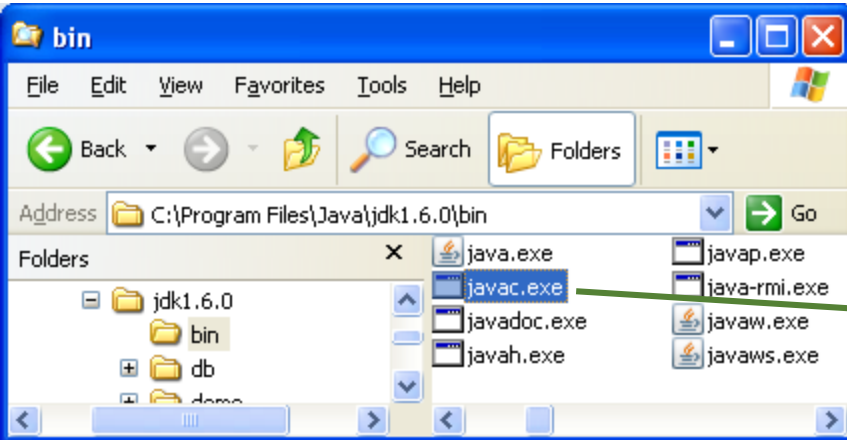
- What can Java Technology do?
- How can Java support platform-independence?
- Java Platform
- Set up Environment Variables
- The first Java program in the NetBeans
- Structure of a Java program.
- End users run Java Programs

What can Java Technology do?

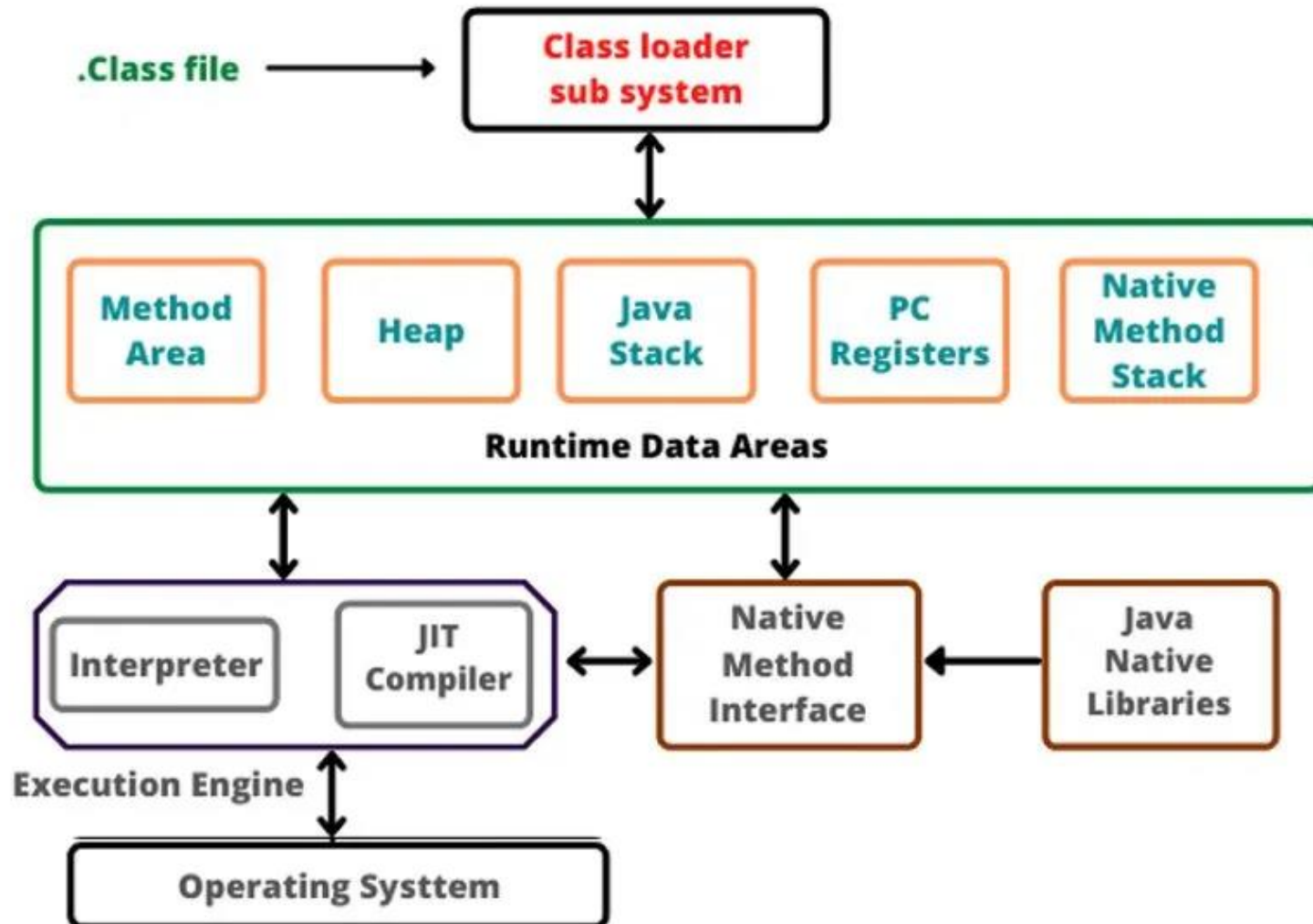
Using Java, we can:

- Development Tools.
- Application Programming Interface (API).
- Deployment Technologies.
- User Interface Toolkits.
- Integration Libraries.
 - Desktop Application (Console App, GUI Apps)
 - Web-based Applications
 - Network-based Applications
 - Game
 - Distributed Applications
 - Embedding Application (Apps on Devices)

How can Java support platform-independence?

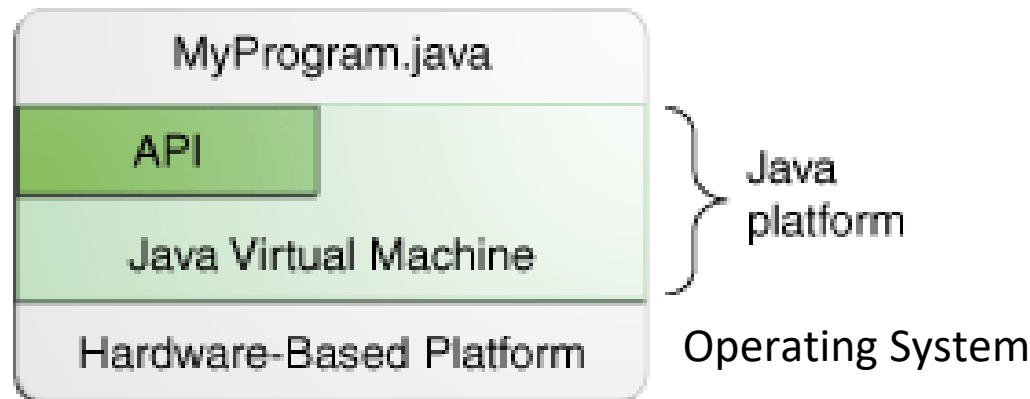


Java Virtual Machine



Java Platform

- A platform is the hardware or software environment in which a program runs.
 - The Java platform has two components:
 - The Java Virtual Machine
 - The **J**ava **A**pplication **P**rogramming Interface (API)



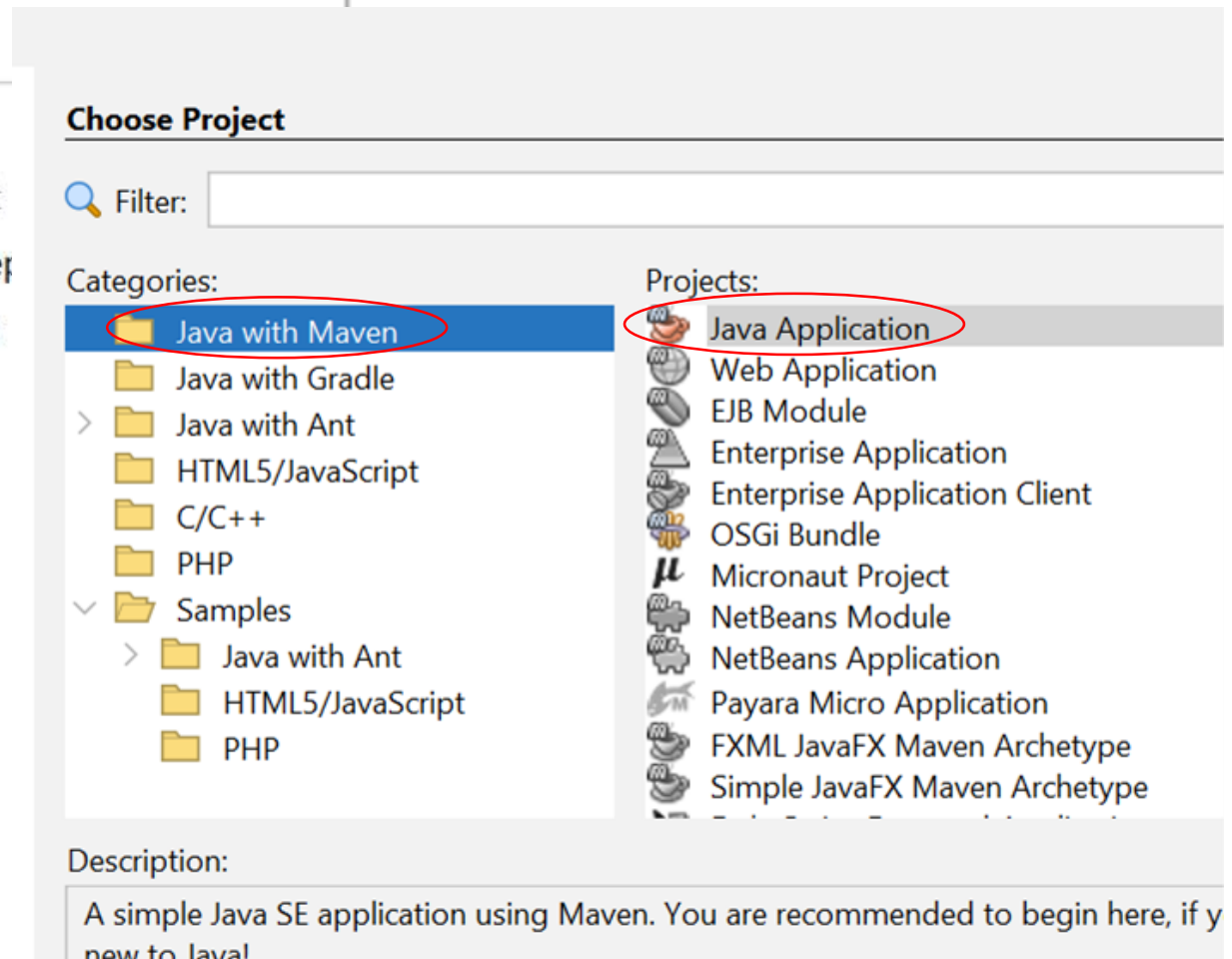
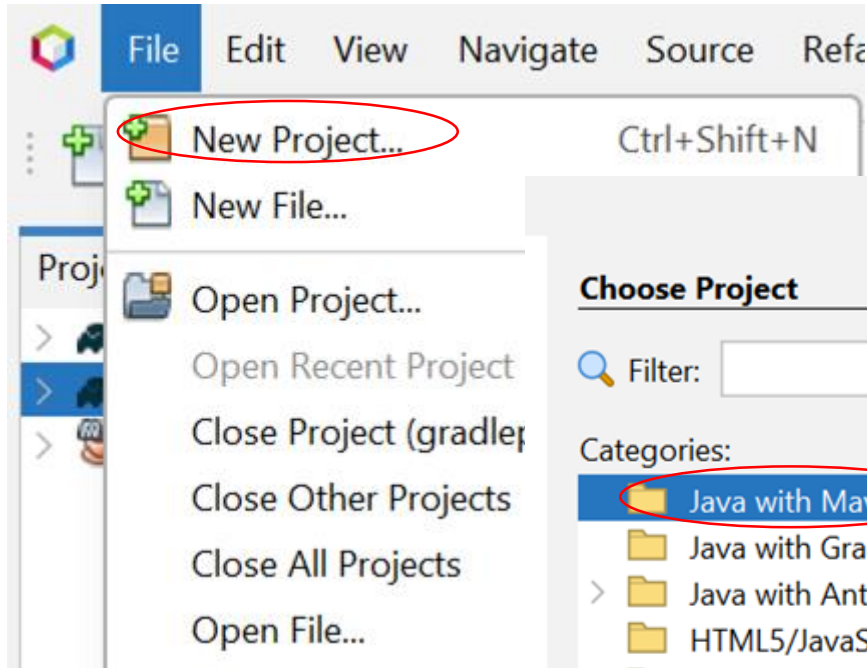
The first Java program in the NetBeans

This program will show the string “Hello World” to the screen.

Steps

- 1- Create a new Java NetBeans project
- 2- Add a Java class
- 3- Write code
- 4- Compile/Run the program

New Project



New Project

New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name: HelloWorld

Project Location: D:\PTKNGON\TEACHING\OOP_OOAD\Demo Browse...

Project Folder: D:\PTKNGON\TEACHING\OOP_OOAD\Demo\HelloWorld

Artifact Id: HelloWorld

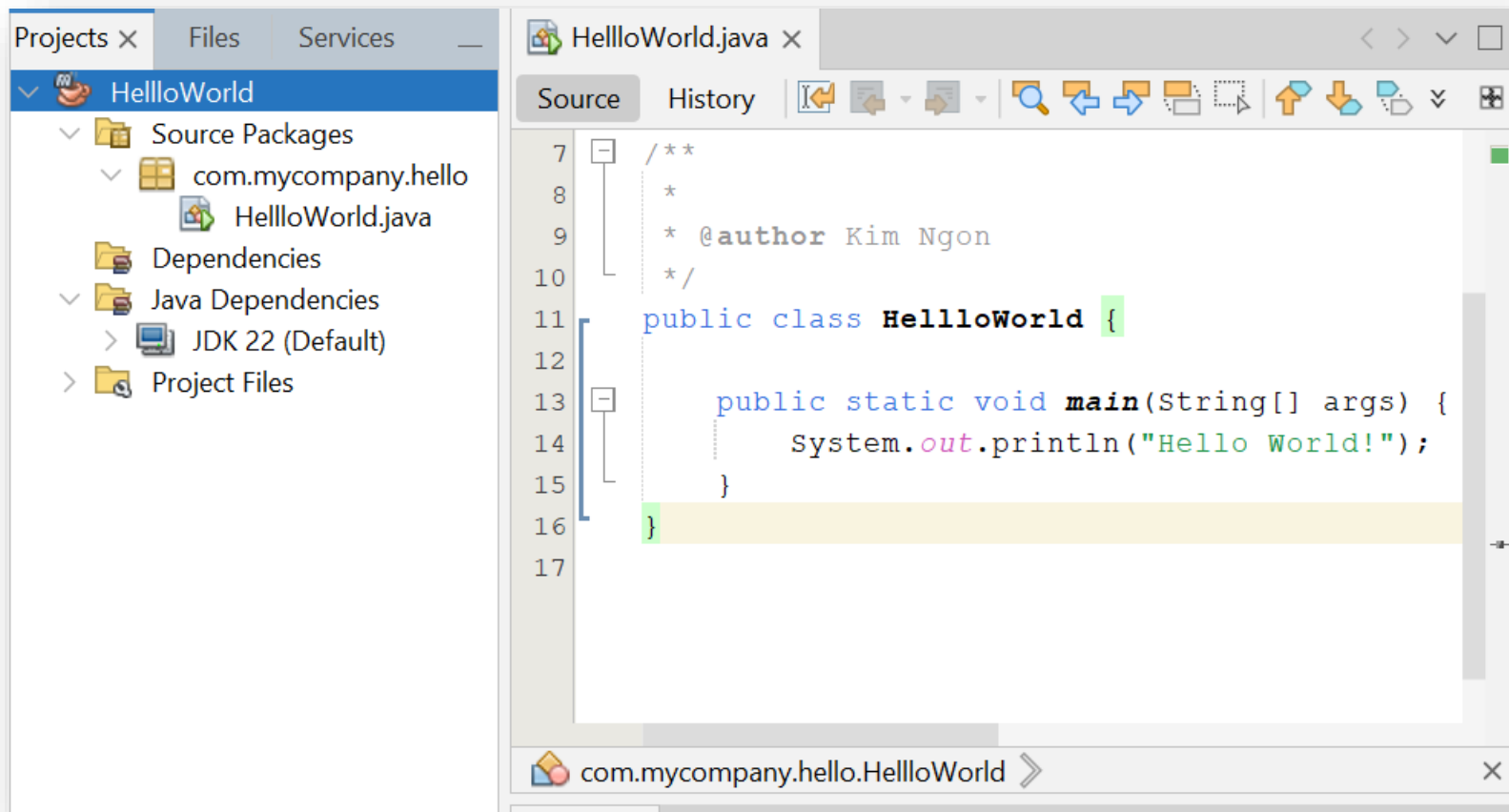
Group Id: com.mycompany

Version: 1.0-SNAPSHOT

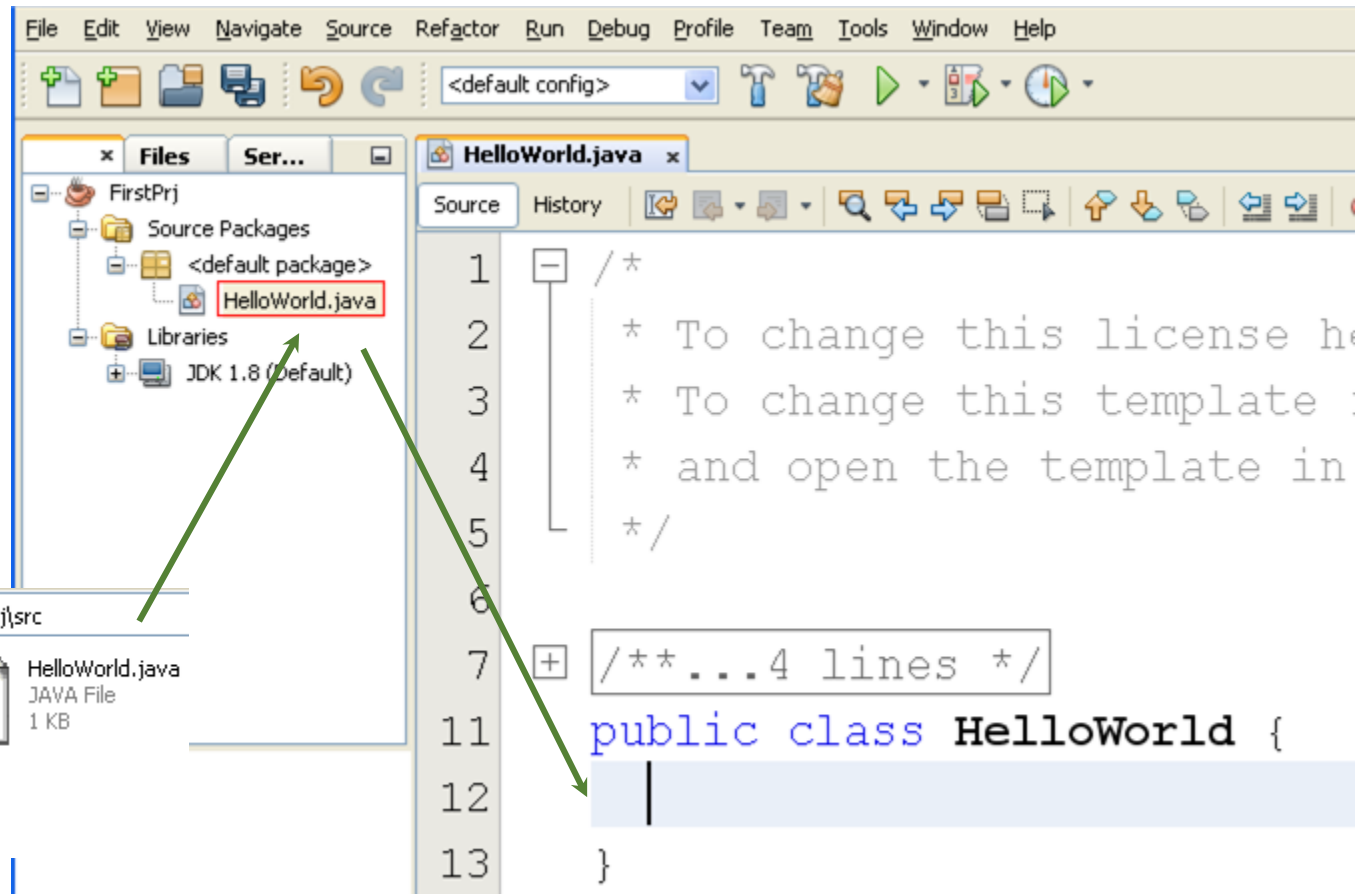
Package: com.mycompany.hello (Optional)

< Back Next > Finish Cancel Help

Initial Project Structure



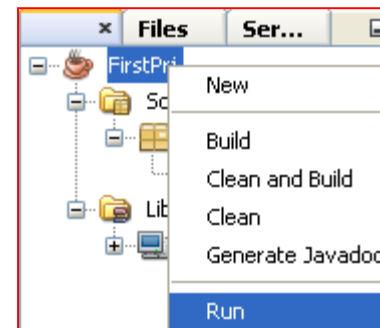
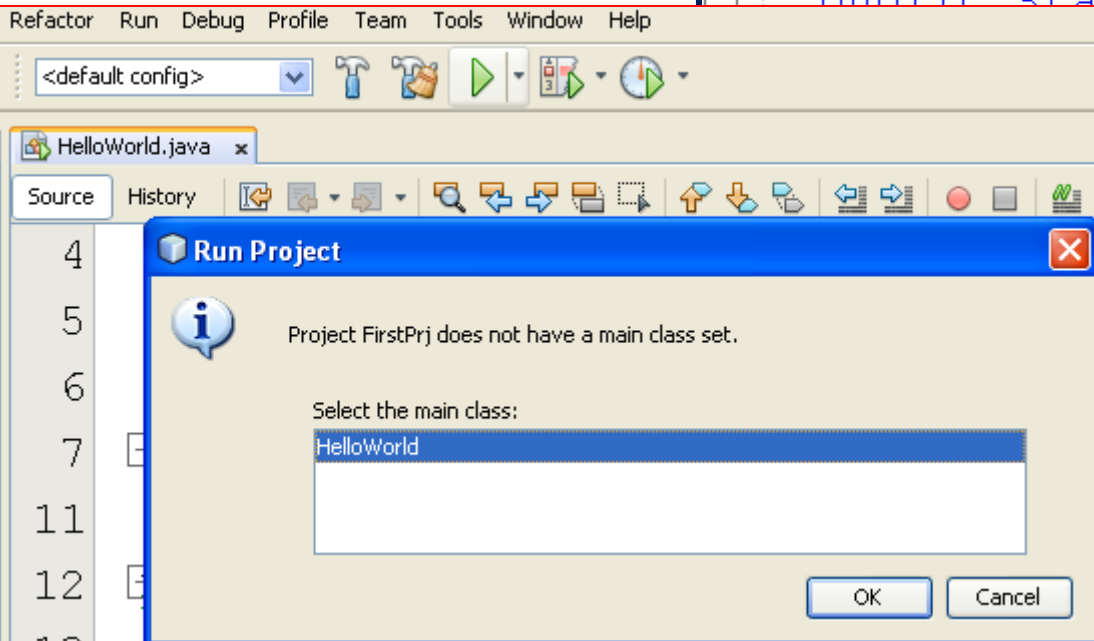
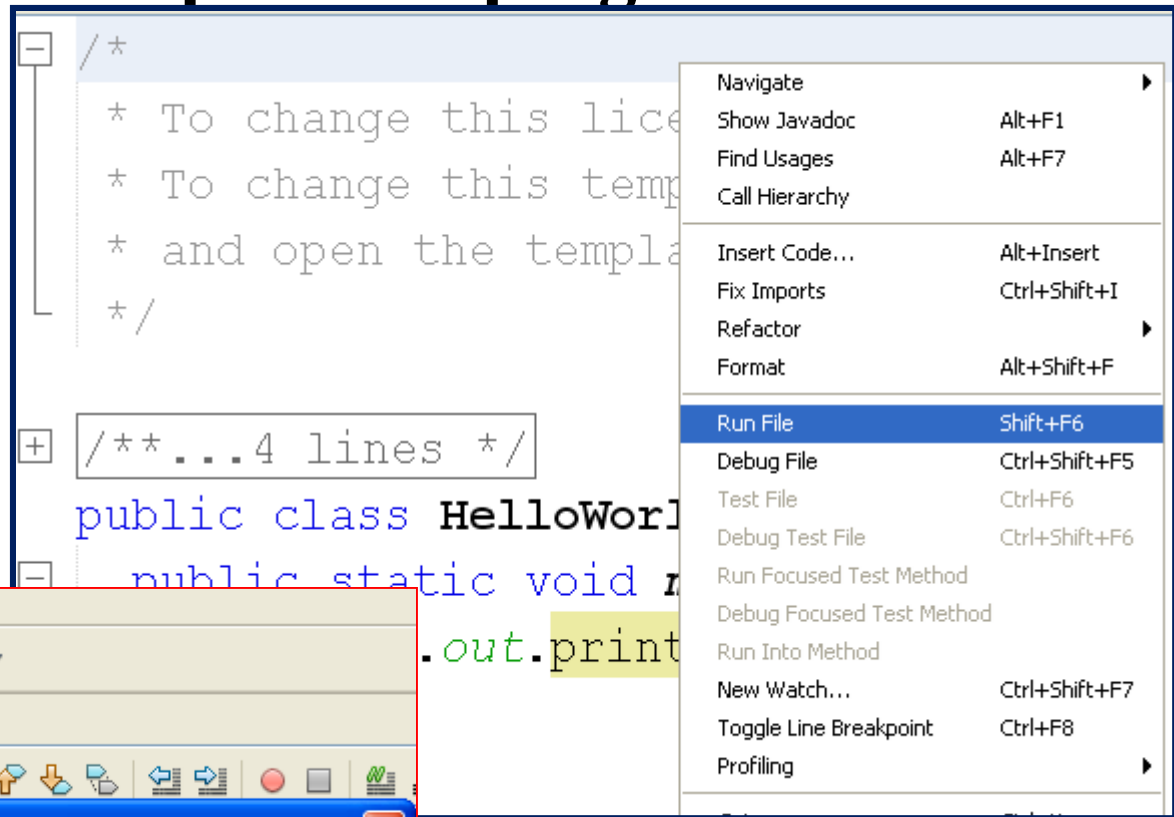
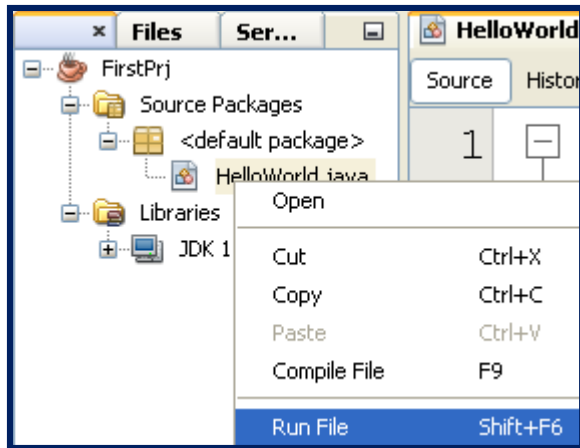
Add a Java Class...



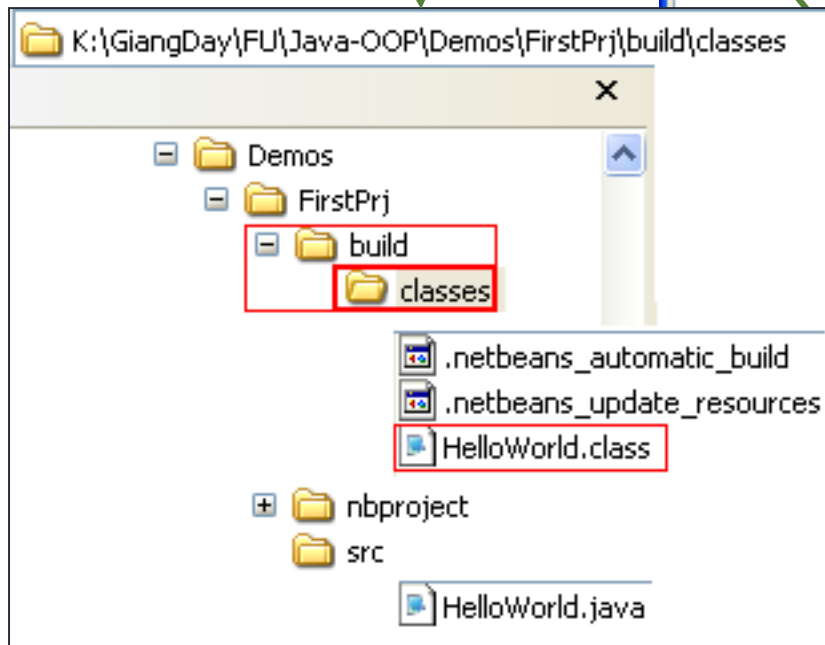
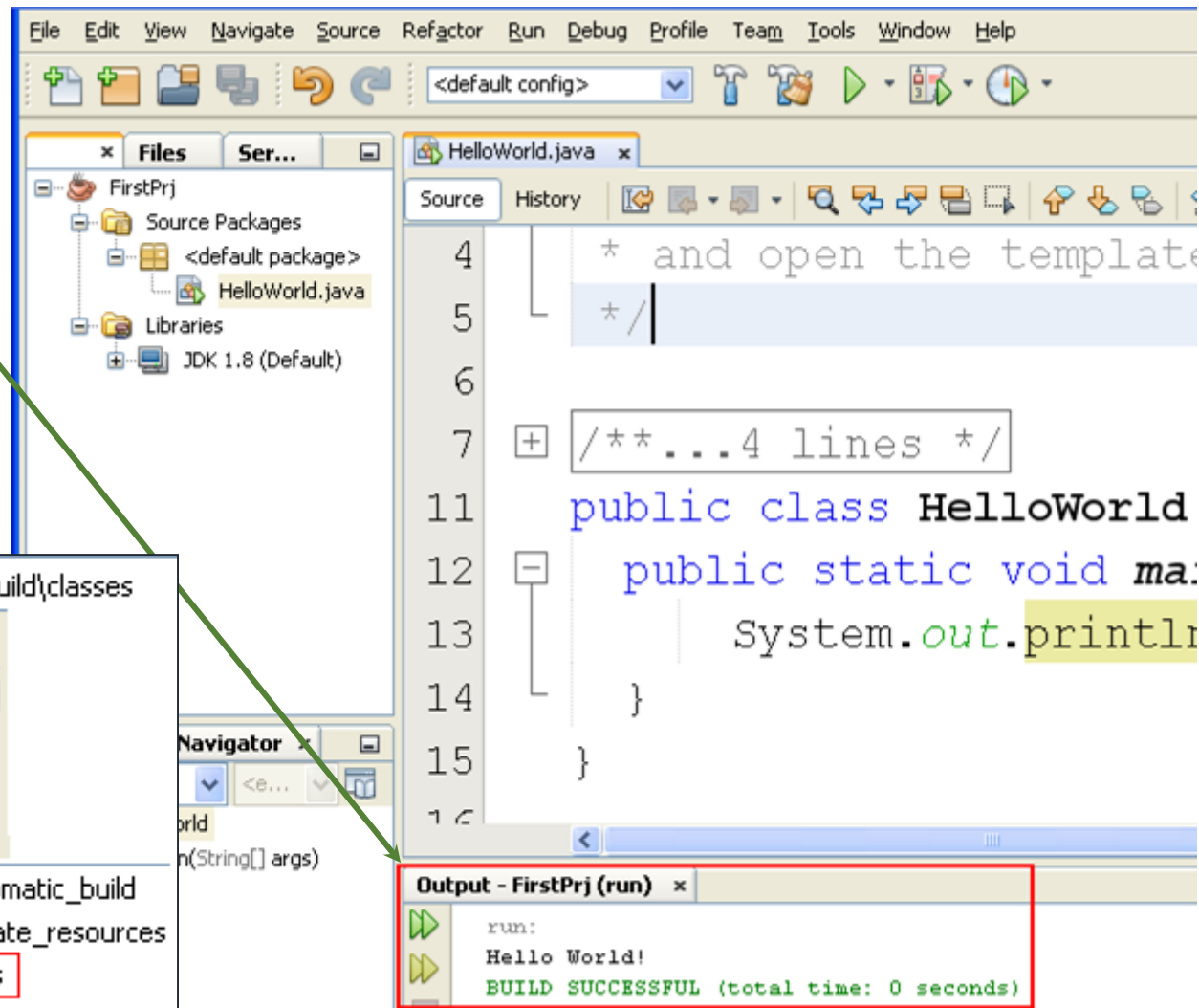
In Windows Explorer

In NetBeans

Step 4: 4 ways to Compile/Run program in NetBeans



Result:



Explain JDK and its tools

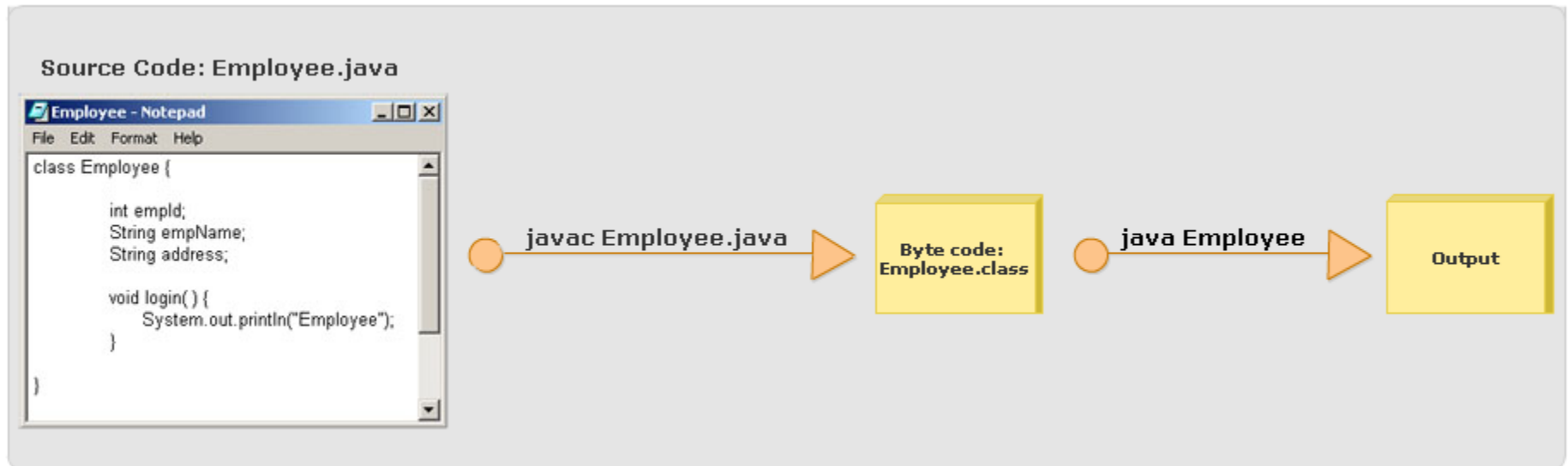
- javac (Java compiler)

```
javac [option] source
```

where,
`source` is one or more file names that end with the extension `.java`.
- java (Java interpreter)

```
java [option] classname [arguments]
```

where,
`classname` is the name of the class file.



A Closer Look at the "Hello World!" Application

- *Comments*

- Traditional **`/*this is a comment*/`**
- Comment to line end **`//this is an end of line comment`**

- *Class declaration*

- **`public class ClassName { ... }`**
- For example: `public class HelloWorld { ... }`

- *The main Method – Entry point of Java program*

- **`public static void main(String[] args) {..}`**

Learning the Java Language

(<http://docs.oracle.com/javase/tutorial/java/index.html>)

Objectives

- Study some fundamentals of Java languages: Data types, variables, arrays, operators, logic constructs.
- Pass arguments to the main method
- Input/output variables
- Object-Oriented Programming Concepts: Class, Interface, Package.

Keywords and Identifiers

- Keywords: Almost of them are similar to those in C language
- Naming Convention:

Letter	Letters
\$	Digits, \$
—	—

- Java is a case-sensitive language
- Identifiers must be different to keywords

Primitive Data Types - Variables

- A **primitive** is a **simple non-object data type** that represents a single value. Java's primitive data types are:

Type	Bytes	Minimum	Maximum
char	2	\u0000	\uFFFF
byte	1	-2^7	$2^7 - 1$
short	2	-2^{15}	$2^{15} - 1$
int	4	-2^{31}	$2^{31} - 1$
long	8	-2^{63}	$2^{63} - 1$
float	4		
double	8		
boolean	true/false		

Type var [=Initial value] ;

Operators

Category (Descending Precedence)	Operators
Unary	++ -- + - ! ~ (type)
Arithmetic	* / % + -
Shift	<< >> >>>
Comparison	< <= > >= instanceof == !=
Bitwise	& ^
Short-circuit	&&
Conditional	?:
Assignment	= op=

They are the same with
those in C language

Literals and Value Variables

- Character: 'a'
- String: String S="Hello";
- Escape sequences: see the page 10
- Integral literals:
 - 28, 0x1c, 0X1A (default: int). 123l, 123L (long)
- Floating point:
 - 1.234 (default: double)
 - 1.3f 1.3F
 - 1.3E+21
 - 1.3d 1.3D

Value variable

Stack

n

10

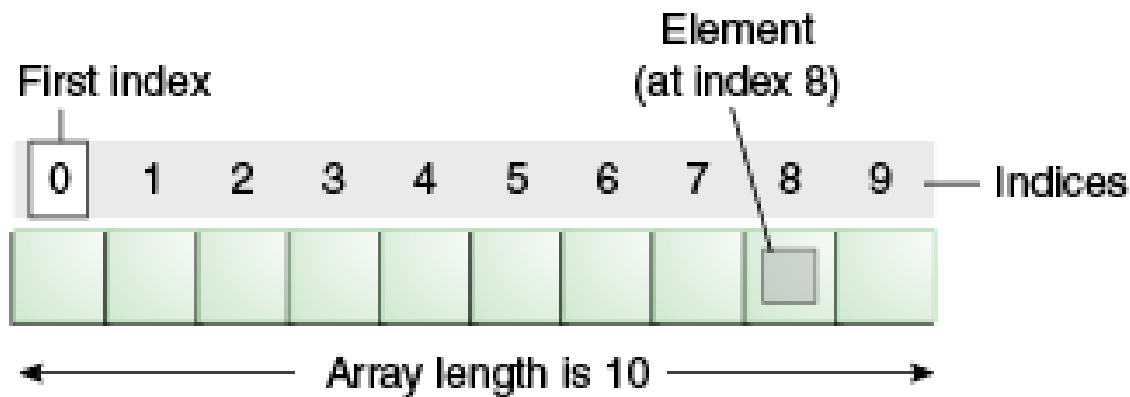
int n=10;

Java Expressions

- Java is an expression-oriented language. A simple expression in Java is either:
 - A constant: 7, false
 - A char - literal enclosed in single quotes: 'A', '3'
 - A String - literal enclosed in double quotes: "foo"
 - The name of any properly declared variables: x
 - Any two|one of the preceding types of expression that are combined with one of the Java binary operators: i++, x + 2, (x + 2)

One Dimensional Arrays (1)

- An *array* is a container object that holds a fixed number of values of a single type.
- The length of an array is established when the array is created.
- Each item in an array is called an *element*, and each element is accessed by its numerical *index*.



One Dimensional Arrays (2)

- Declaring a Variable to Refer to an Array

```
int[] anArray;
```

```
or float anArrayOfFloats[];
```

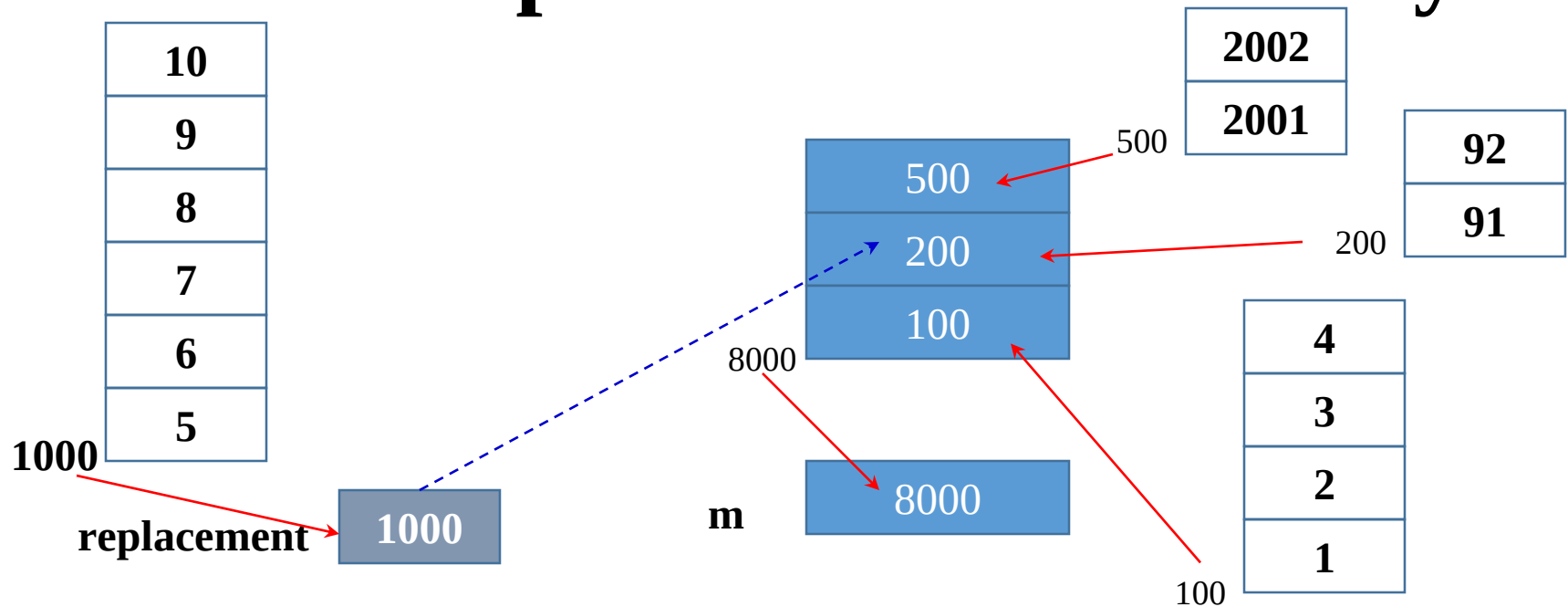
- Creating, Initializing, and Accessing an Array

```
anArray = new int[10];
```

- Copying Arrays

- Use arraycopy method from System class.

Multiple Dimensional Arrays



```
int m[][]= { {1,2,3,4}, {91,92}, {2001,2002}};
```

```
int[] replacement = {5,6,7,8,9,10};
```

```
m[1]= replacement;
```

`m[i][j]`

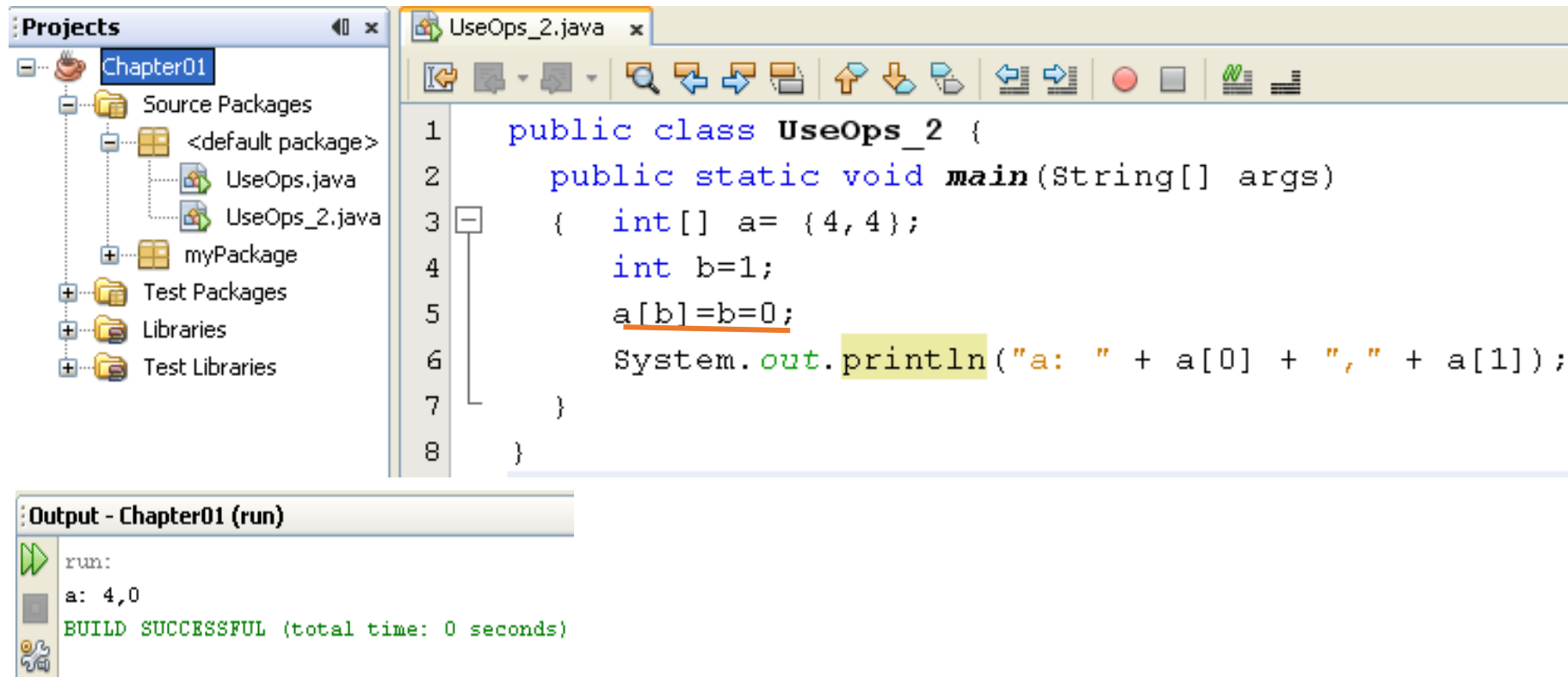
```
int[][] m; // declare a matrix
int r=10, c=5; // number of rows, columns
m= new int[r][c]; // memory allocate
```

Evaluating Expressions and Operator Precedence

- The compiler generally evaluates such expressions from the innermost to outermost parentheses, left to right.

```
int x = 1; int y = 2; int z = 3;  
int answer = ((8 * (y + z)) + y) * x;  
would be evaluated piece by piece as follows:  
((8 * (y + z) ) + y) * x  
((8 * 5) + y) * x  
(40 + y) * x  
42 * x  
42
```

Operator Precedence- Evaluation Order



The screenshot displays an IDE with the following components:

- Projects Panel:** Shows a project named 'Chapter01' containing 'Source Packages' (with '<default package>' and 'myPackage'), 'Test Packages', 'Libraries', and 'Test Libraries'. The files 'UseOps.java' and 'UseOps_2.java' are listed under the default package.
- Editor:** Displays the code for 'UseOps_2.java'. The code is as follows:

```
1 public class UseOps_2 {  
2     public static void main(String[] args)  
3     {  
4         int[] a= {4,4};  
5         int b=1;  
6         a[b]=b=0;  
7         System.out.println("a: " + a[0] + "," + a[1]);  
8     }  
}
```
- Output Panel:** Titled 'Output - Chapter01 (run)', it shows the execution results:

```
run:  
a: 4,0  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Basic Constructs

- They are taken from C-language
- Selection
 - if, if ... else
 - switch (char/int exp)... case ... default...
- Loops
 - for
 - do... while
 - while

Basic Logic Constructs

□ They are the same with those in C-statements

An enhanced for loop

```
2 package com;
3 import java.lang.*;
4 public class Chao {
5     public static void main(String args[]) {
6         System.out.println("Hello");
7         int a[] = { 1,2,3,4,5};
8         for (int i=0;i<a.length;i++) System.out.print(a[i] + ",");
9         System.out.println();
10        for (int x : a) System.out.print(x + ",");
11        System.out.println();
12        for (int x : a) x+=10;
13        for (int i=0;i<a.length;i++) System.out.print(a[i] + ",");
14        System.out.println();
15    }
16 }
```

Read only

a

1	2	3	4	5
---	---	---	---	---

x

1

Output - P1 (run)

run:
Hello
1,2,3,4,5,
1,2,3,4,5,
1,2,3,4,5,

The String type

- A String represents a sequence of zero or more Unicode characters.
 - `String name = "Steve";`
 - `String s = "";`
 - `String s = null;`
- String concatenation.
 - `String x = "foo" + "bar" + "!";`
- Java is a case-sensitive language.

Type Conversions and Explicit Casting

```
Casting_Convert_1.java * x
public class Casting_Convert_1 {
    public static void main (String[] args)
    {
        short x, y = 256;
        byte m, n = 6;
        x = n ; // Systematic Conversion
        n = y; // narrow conversion
        n = (byte) y; // narrow casting, possible loss of precision
        System.out.println(n);
    }
}
```

- * Widening Conversion: OK
- Narrowing conversion: Not allowed. We must use explicit casting.
- A boolean can not be converted to any other type.
- A non-boolean can be converted to another non-boolean type.

```
Casting_Convert_1.java * x
public class Casting_Convert_1 {
    public static void main (String[] args)
    {
        short x, y = 256;
        byte m, n = 6;
        x = n ; // Systematic Conversion
        n = (byte) y; // narrow casting, possible loss of p
        System.out.println(n);
    }
}
```

Output - Chapter04 (run)

```
run:
0
BUILD SUCCESSFUL (total time: 0 seconds)
```

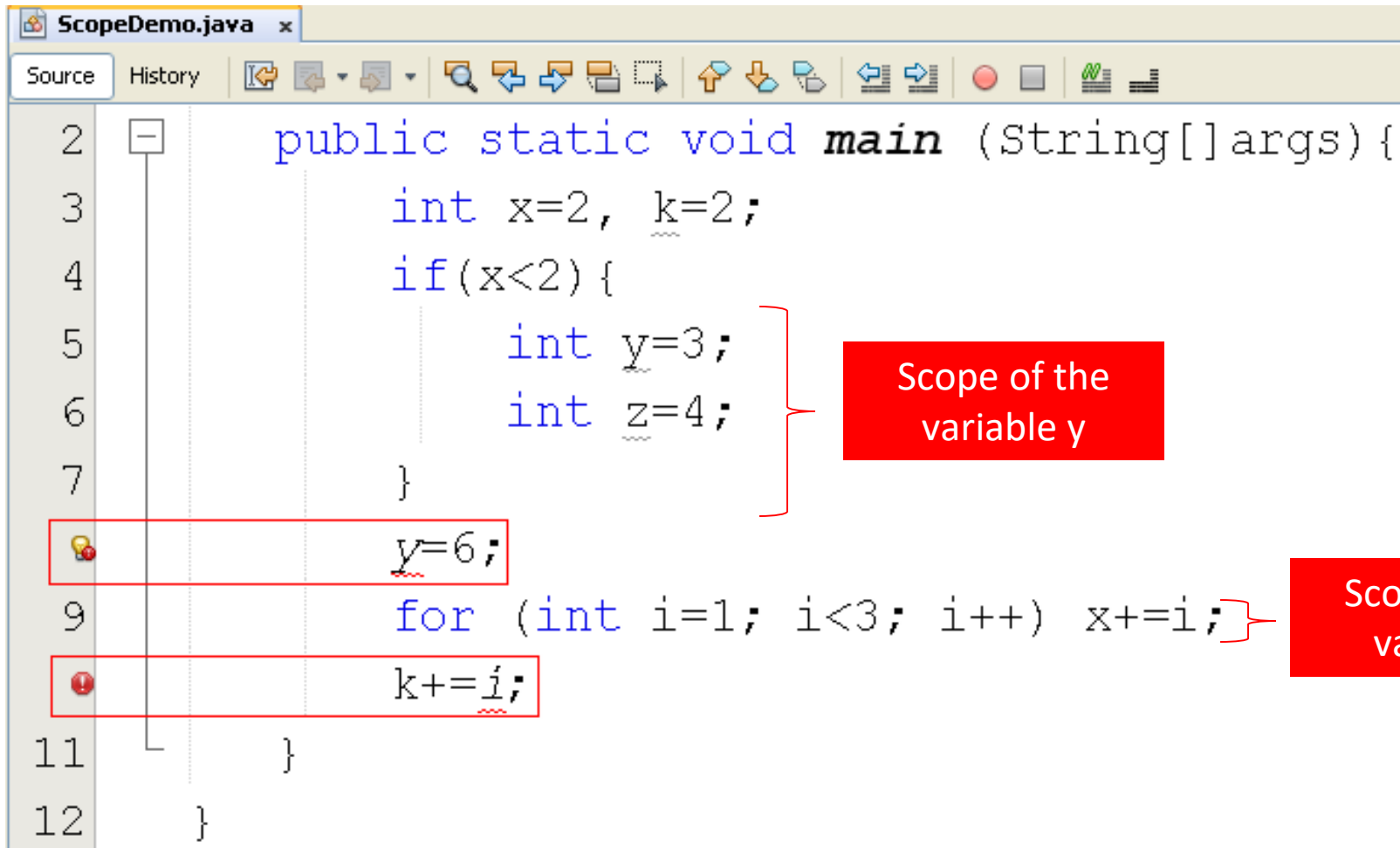
0000 0001

0000 0000

y

n

Scope of a Variable



The screenshot shows a code editor window titled "ScopeDemo.java". The code is as follows:

```
2 public static void main (String[] args) {  
3     int x=2, k=2;  
4     if(x<2) {  
5         int y=3;  
6         int z=4;  
7     }  
8     y=6;  
9     for (int i=1; i<3; i++) x+=i;  
10    k+=i;  
11 }  
12 }
```

Annotations in the image:

- A red bracket on the right side of lines 5 and 6 points to a red box containing the text "Scope of the variable y".
- A red box highlights line 8, containing the code `y=6;`. A red bracket on the right side of lines 8 and 9 points to a red box containing the text "Scope of the variable i".
- A red box highlights line 10, containing the code `k+=i;`. The variable `i` in this line is underlined in red.

Input/Output Data

Class java.lang.System

Class java.util.Scanner

Refer to Java documentation:
java.lang.String class,
- the **format** method,
- format string
for more details

```
InputOutputDemo.java x
1  /* Write a program that will accept an array of integers then
2     print out entered value and the sum of values
3  */
4  import java.util.Scanner;
5  public class InputOutputDemo {
6      public static void main (String args[])
7      {
8          int a[]; // array of integers
9          int n; // number of elements of the array
10         int i; // variable for traversing the array
11         Scanner sc= new Scanner(System.in); // object for the keyboard
12         System.out.print("Enter number of elements: ");
13         n = Integer.parseInt(sc.nextLine());
14         a = new int[n]; // mem. allocating for elements of the array
15         for (i=0;i<n;i++)
16         {
17             System.out.print("Enter the " + (i+1) + "/" + n + " element: ");
18             a[i]=Integer.parseInt(sc.nextLine());
19         }
20         System.out.print("Entered values: ");
21         for (i=0;i<n;i++) System.out.format("%5d", a[i]);
22         int S=0;
23         for (int x: a) S+=x;
24         System.out.println("\nSum of values: " + S);
25     }
26 }
```

n= sc.nextInt();

```
Output - Chapter01 (run) #2
run:
Enter number of elements: 5
Enter the 1/5 element: 1
Enter the 2/5 element: 4
Enter the 3/5 element: 2
Enter the 4/5 element: 0
Enter the 5/5 element: 7
Entered values:      1      4      2      0      7
Sum of values: 14
BUILD SUCCESSFUL (total time: 11 seconds)
```

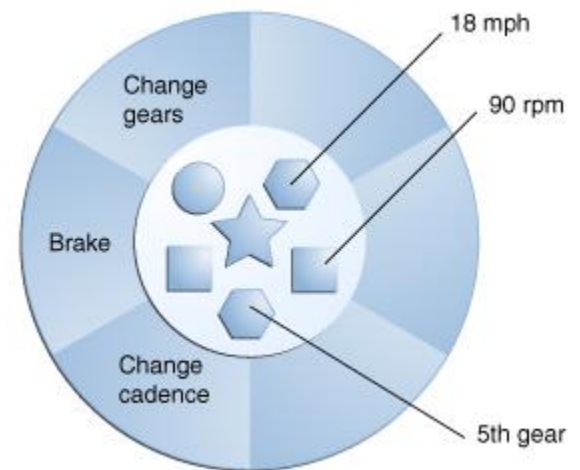
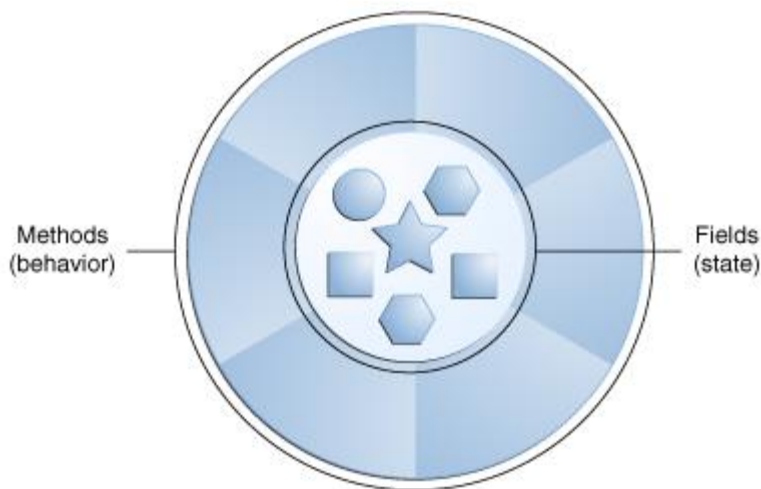
What Is an Object?(1)



- Objects are key to understanding *object-oriented* technology.
- Examples of real-world objects: your dog, your desk, your television set, your bicycle.
- Real-world objects share two characteristics: They all have *state* and *behavior*.
 - Dogs have **state** (name, color, breed, hungry) and **behavior** (barking, fetching, wagging tail).

What Is an Object?(2)

- Software objects are conceptually similar to real-world objects: they too consist of state and related behavior.
- An object stores its state in *fields* and exposes its behavior through *methods*.



What Is an Object?(3)

- Software objects provides a number of benefits:
 - Modularity
 - Information-hiding
 - Code re-use
 - Pluggability and debugging ease

```
class Bicycle {  
  
    int cadence = 0;  
    int speed = 0;  
    int gear = 1;  
  
    void changeCadence(int newValue) {  
        cadence = newValue;  
    }  
  
    void changeGear(int newValue) {  
        gear = newValue;  
    }  
  
    void speedUp(int increment) {  
        speed = speed + increment;  
    }  
  
    void applyBrakes(int decrement) {  
        speed = speed - decrement;  
    }  
  
    void printStates() {  
        System.out.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }  
}
```

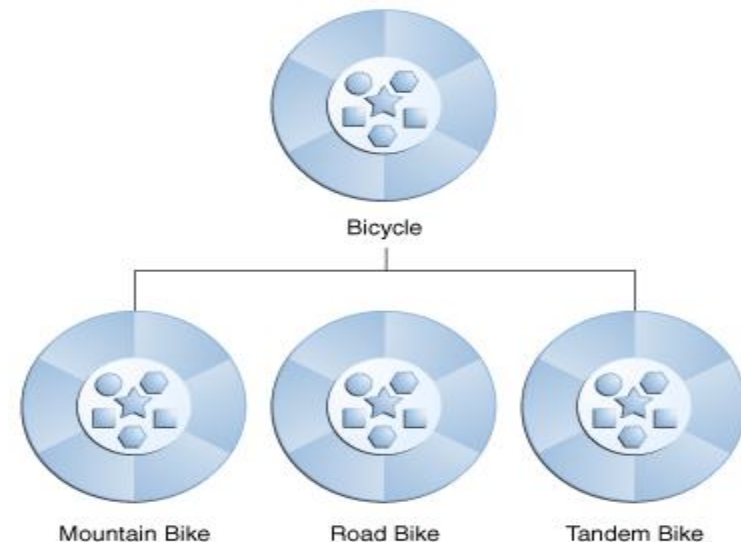
What Is a Class?

- A *class* is the blueprint from which individual objects are created.
 - Your bicycle is an *instance* of the *class* of *objects* known as bicycles.

What Is Inheritance?

- Object-oriented programming allows classes to *inherit* commonly used state and behavior from other classes.
 - Bicycle now becomes the *superclass* of MountainBike, RoadBike, and TandemBike

```
class MountainBike  
  
extends Bicycle {  
  
    // new fields and methods defining  
    // a mountain bike would go here  
}
```



What Is an Interface?

- An interface is a group of related methods with empty bodies.

```
interface Bicycle {  
    // wheel revolutions per minute  
    void changeCadence(int newValue);  
    void changeGear(int newValue);  
    void speedUp(int increment);  
    void applyBrakes(int decrement);  
}
```


What Is a Package?

- A package is a namespace that organizes a set of related classes and interfaces.
- The Java platform provides an enormous class library (a set of packages) suitable for use in your own applications called API.
 - For example, a String object contains state and behavior for character strings.

User-Defined Package

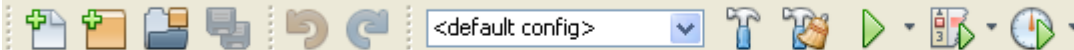
- Add a Java class

New Java Class

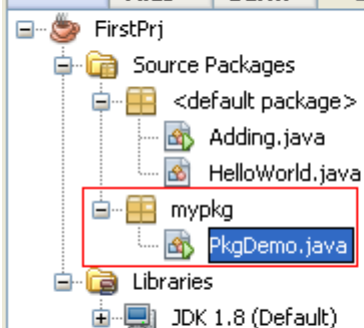
Steps	Name and Location
1. Choose File Type	Class Name: PkgDemo
2. Name and Location	Project: FirstPrj
	Location: Source Packages
	Package: mypkg
	Created File: K:\GiangDay\FU\Java-OOP\Demos\FirstPrj\src\mypkg\PkgDemo.java

FirstPrj - NetBeans IDE 8.0.2

File Edit View Navigate Source Refactor Run



Files Ser...



Adding.java x PkgDemo.java x

Source History

```
1 package mypkg;  
2 public class PkgDemo {  
3     public static void main(String[] args){  
4         System.out.println("Package Demo.");  
5     }  
6 }  
7
```

If package is used, it must be the first line in Java code

Output - FirstPrj (run) x

```
run:  
Package Demo.  
BUILD SUCCESSFUL (total time: 0 seconds)
```

Summary

- The core concepts behind object-oriented programming: objects, interfaces, classes, and inheritance.
- The traditional features of the language, including variables, arrays, data types, operators, and control flow.